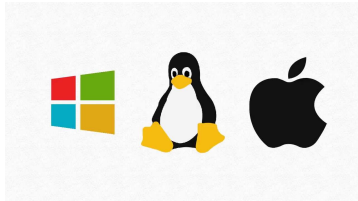


컴퓨터 시스템

Scheduling



This material is reproduced based on
핵심 컴퓨터구조 및 운영체제 해설(최종언저, 예문사),
쉽게 배우는 운영체제(조성호, 한빛아카데미, 2018)
Copyrights are reserved by corresponding authors.

CPU 스케줄링

가. CPU 스케줄링의 정의

- Process 작업수행을 위해 언제, 어느 Process에 CPU를 할당할 것인지를 결정하는 작업
- Multi-Processor 환경 하에서 Processor 간의 우선순위를 저장함으로써 CPU 활용을 극대화하기 위한 방법

나. CPU 스케줄링의 기준

- 바람직한 동작을 보이는 프로세스에게 더 좋은 서비스 제공
- I/O Bound와 CPU Bound 프로세스의 적절한 혼용
- Process의 작업형태, 우선순위, Burst 시간, 잔여실행시간을 복합적으로 고려

: -> ps -> ?

© 2021 Computer System

스케줄링의 단계

■ 고수준 스케줄링

- 시스템 내의 전체 작업 수를 조절하는 것
- 어떤 작업을 시스템이 받아들일지 또는 거부할지를 결정
- 시스템 내에서 동시에 실행 가능한 프로세스의 총 개수가 정해짐
- 장기 스케줄링, 작업 스케줄링, 승인 스케줄링이라고도 함

■ 저수준 스케줄링

- 어떤 프로세스에 CPU를 할당할지, 어떤 프로세스를 대기 상태로 보낼지 등을 결정
- 아주 짧은 시간에 일어나기 때문에 단기 스케줄링이라고도 함

© 2021 Computer System

스케줄링의 단계

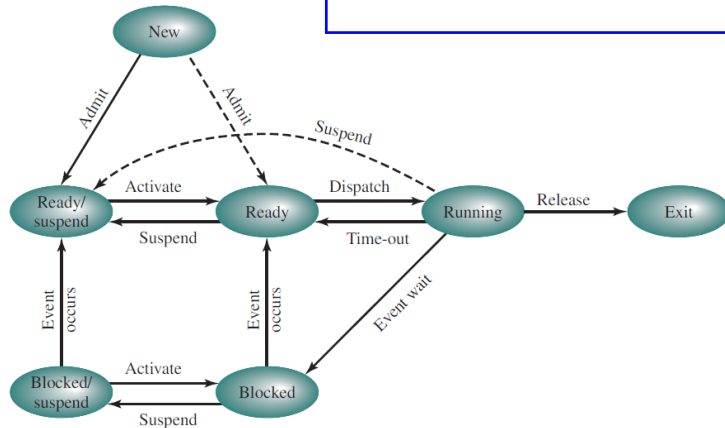
■ 중간 수준 스케줄링

- 중지와 활성화로 전체 시스템의 활성화된 프로세스 수를 조절하여 과부하를 막음
- 일부 프로세스를 중지 상태로 옮김으로써 나머지 프로세스가 원만하게 작동하도록 지원
- 저수준 스케줄링이 원만하게 이루어지도록 완충하는 역할

© 2021 Computer System

Process State and Scheduling

2 1) : ->blocked 2) : ok x, ready * block -> ready



© 2021 Computer System

스케줄링의 단계

- 정리

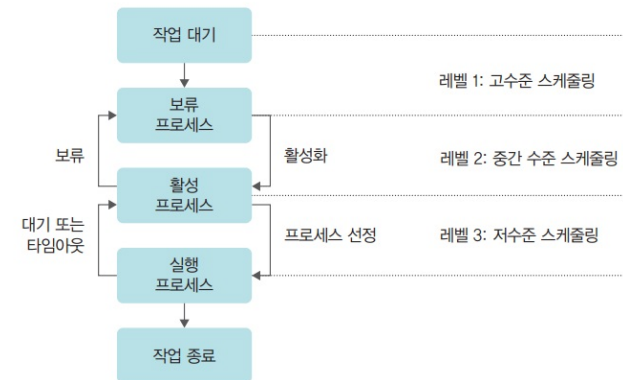


그림 4-3 스케줄링의 단계

© 2021 Computer System

CPU 스케줄링 요건

요건	설명
처리능력(Throughput) 최대화	주어진 시간에 최대한 많은 작업 처리(처리된 프로세스 수/시간)
CPU 이용률(Utilization) 극대화	CPU를 정해진 시간 내 100% 가동
Fairness 최대화	프로세스별 CPU 자원 할당의 공정성
Deadline 최대화	실시간 환경 등에서의 즉시처리 한계시간
반환(경과)시간(Turnaround time) 최소화	전체작업 수행시간을 최소화(system in → system out 시간)
대기시간(Waiting time) 최소화	ready queue에서 기다리는 시간 최소화
응답시간(Response time) 최소화	대화식 시스템에서 첫 응답까지의 시간 최소화

© 2021 Computer System

스케줄링의 목적

공평성 fairness	모든 프로세스가 자원을 공평하게 배정받아야 하며, 자원 배정 과정에서 특정 프로세스가 배제되어서는 안 됨
효율성 efficient	시스템 자원이 유휴 시간 없이 사용되도록 스케줄링을 하고, 유휴 자원을 사용하는 프로세스에는 우선권을 주어야 함
안정성 stability	우선순위를 사용하여 중요 프로세스가 먼저 작동하도록 배정함으로써 시스템 자원을 점유하거나 파괴하려는 프로세스로부터 자원을 보호해야 함
확장성	프로세스가 증가해도 시스템이 안정적으로 작동하도록 조치해야 하며 시스템 자원이 늘어나는 경우 이 혜택이 시스템에 반영되게 해야 함
반응 시간 보장	응답이 없는 경우 사용자는 시스템이 멈춘 것으로 가정하기 때문에 시스템은 적절한 시간 안에 프로세스의 요구에 반응해야 함
starvation prevention	특정 프로세스의 작업이 무한히 연기되어서는 안 됨

response time

starvation prevention

© 2021 Computer System

CPU 집중 프로세스와 입출력 집중 프로세스

■ CPU 집중 프로세스

- 수학 연산과 같이 CPU를 많이 사용하는 프로세스로 CPU 버스트가 많은 프로세스

■ 입출력 집중 프로세스

- 저장장치에서 데이터를 복사하는 일과 같이 입출력을 많이 사용하는 프로세스로 입출력 버스트가 많은 프로세스

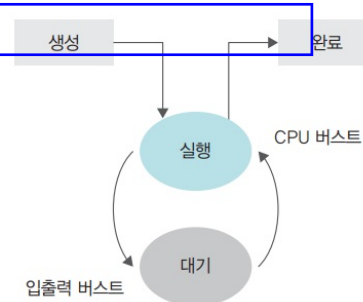


그림 4-5 CPU 버스트와 입출력 버스트

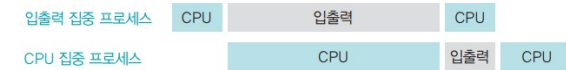
CPU 집중 프로세스와 입출력 집중 프로세스

■ 우선 배정

- 스케줄링을 할 때 입출력 집중 프로세스의 우선순위를 CPU 집중 프로세스보다 높이면 시스템의 효율이 향상



(a) CPU 집중 프로세스 우선 배정



(b) 입출력 집중 프로세스 우선 배정

그림 4-6 CPU 집중 프로세스와 입출력 집중 프로세스의 우선 배정 결과

정리

■ CPU 스케줄링 시 고려사항

선점형 스케줄링과 비선점형 스케줄링

■ 선점형 스케줄링(Preemptive Scheduling)

- 운영체제가 필요하다고 판단하면 실행 상태에 있는 프로세스의 작업을 중단시키고 새로운 작업을 시작할 수 있는 방식
- 하나의 프로세스가 CPU를 독점할 수 없기 때문에 빠른 응답 시간을 요구하는 대화형 시스템이나 시분할 시스템에 적합
- 대부분의 저수준 스케줄러는 선점형 스케줄링 방식을 사용

■ 비선점형 스케줄링(Non-preemptive Scheduling)

- 어떤 프로세스가 실행 상태에 들어가 CPU를 사용하면 그 프로세스가 종료되거나 자발적으로 대기 상태에 들어가기 전까지는 계속 실행되는 방식
- 선점형 스케줄링보다 스케줄러의 작업량이 적고 문맥 교환에 의한 낭비도 적음
- CPU 사용 시간이 긴 프로세스 때문에 CPU 사용 시간이 짧은 여러 프로세스가 오랫동안 기다리게 되어 전체 시스템의 처리율이 떨어짐
- 과거의 일괄 작업 시스템에서 사용하던 방식

스케줄링 알고리즘의 선택 기준

스케줄링 알고리즘의 평가 기준

- CPU 사용률
 - 전체 시스템의 동작 시간 중 CPU가 사용된 시간을 측정하는 방법
 - 가장 이상적인 수치는 100%이지만 실제로는 여러 가지 이유로 90%에도 못 미침
- 처리량
 - 단위 시간당 작업을 마친 프로세스의 수
 - 이 수치가 클수록 좋은 알고리즘임

© 2021 Computer System

스케줄링 알고리즘의 선택 기준

스케줄링 알고리즘의 평가 기준

- 대기 시간 :
- 응답 시간 :
- 실행 시간 :
- 반환 시간 :

© 2021 Computer System

스케줄링 알고리즘의 선택 기준

평균 대기 시간

- 모든 프로세스의 대기 시간을 합한 뒤 프로세스의 수로 나눈 값

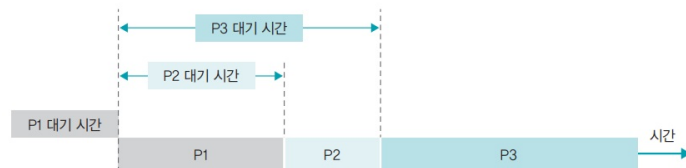


그림 4-13 평균 대기 시간

© 2021 Computer System

스케줄링 기법

구분	선점(Preemptive) 스케줄링	비선점(Non-preemptive) 스케줄링
개념	• 한 프로세스가 CPU를 차지하고 있을 때 우선순위가 높은 다른 프로세스가 현재 프로세스를 중지시키고 자신이 CPU를 점유 가능 • 높은 우선순위를 가진 프로세스들이 빠른 처리를 요구하는 시스템에서 유용	• 한 프로세스가 CPU를 할당받으면 작업 종료 후 CPU 반환 시까지 다른 프로세스는 CPU 점유 불가
장점	• 비교적 빠른 응답 • 대화식 시분할 시스템에 적합	• 응답시간 예측이 용이 • 모든 프로세스에 대한 요구를 공정하게 처리
단점	• 높은 우선순위 프로세스들이 들어오는 경우 오버헤드를 초래	• 짧은 작업을 수행하는 프로세스가 긴 작업 종료 시까지 대기
기법	Round Robin, SRT, 다단계 큐, 다단계 피드백 큐	FCFS, Priority, Deadline, SJF, HRN
활용	실시간 응답환경, Deadline 응답환경	처리시간 편차가 적은 특정 프로세스 환경

© 2021 Computer System

FCFS 스케줄링

FCFS 스케줄링의 동작 방식

- 준비 큐에 도착한 순서대로 CPU를 할당하는 비선점형 방식
- 한 번 실행되면 그 프로세스가 끝나야만 다음 프로세스를 실행할 수 있음
- 큐가 하나라 모든 프로세스는 우선순위가 동일

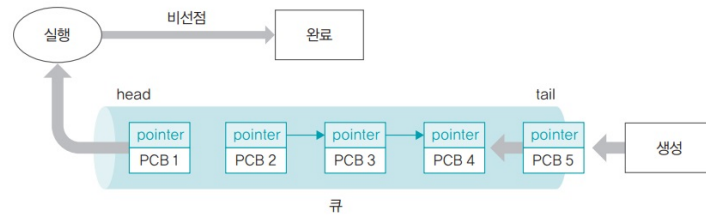


그림 4-14 FCFS 스케줄링의 동작

© 2021 Computer System

FCFS 스케줄링

FCFS 스케줄링의 성능

도착 순서	도착 시간	작업 시간
P1	0	30
P2	3	18
P3	6	9

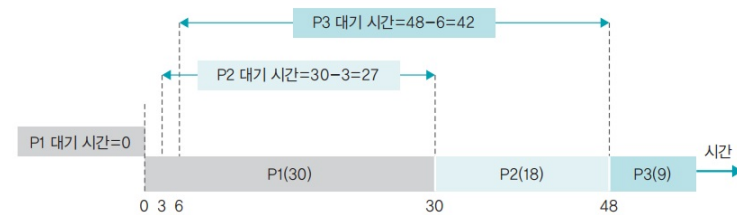


그림 4-15 FCFS 스케줄링의 평균 대기 시간

© 2021 Computer System

FCFS 스케줄링

FCFS 스케줄링의 평가

- 처리 시간이 긴 프로세스가 CPU를 차지하면 다른 프로세스들은 하염없이 기다려 시스템의 효율성이 떨어짐
- 특히 현재 작업 중인 프로세스가 입출력 작업을 요청하는 경우 CPU가 작업하지 않고 쉬는 시간이 많아져 작업 효율이 떨어짐

© 2021 Computer System

SJF(Shortest Job First) 스케줄링

SJF 스케줄링의 동작 방식

- 준비 큐에 있는 프로세스 중에서 실행 시간이 가장 짧은 작업부터 CPU를 할당하는 비선점형 방식
- 최단 작업 우선 스케줄링이라고도 함
- 콘보이 효과를 완화하여 시스템의 효율성을 높임

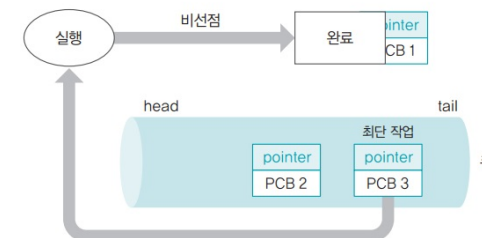


그림 4-17 SJF 스케줄링의 동작

© 2021 Computer System

스케줄링

▪ SJF 스케줄링의 성능

도착 순서	도착 시간	작업 시간
P1	0	30
P2	3	18
P3	6	9

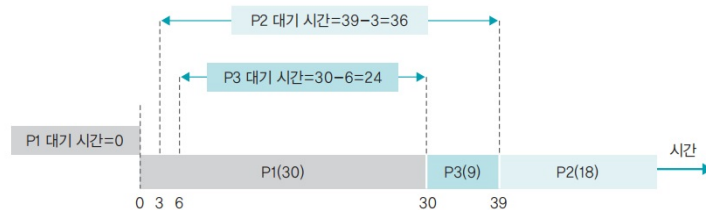


그림 4-18 SJF 스케줄링의 평균 대기 시간

© 2021 Computer System

SJF 스케줄링

▪ SJF 스케줄링의 평가

- 운영체제가 프로세스의 종료 시간을 정확하게 예측하기 어려움
- 작업 시간이 길다는 이유만으로 계속 뒤로 밀려 공정성이 현저히 떨어짐. 이를 아사(starvation) 현상이라 부름

▪ 에이징(나이 먹기)

- 아사 현상의 완화 방법
- 프로세스가 양보할 수 있는 상한선을 정하는 방식
- 프로세스가 자신의 순서를 양보할 때마다 나이를 한 살씩 먹어 최대 몇 살까지 양보하도록 규정하는 것

© 2021 Computer System

HRN(Highest Response Ratio Next) 스케줄링

▪ HRN 스케줄링의 동작 방식

- SJF 스케줄링에서 발생할 수 있는 아사 현상을 해결하기 위해 만들어진 비선점형 알고리즘
- 최고 응답률 우선 스케줄링이라고도 함
- 서비스를 받기 위해 기다린 시간과 CPU 사용 시간을 고려하여 스케줄링을 하는 방식
- 프로세스의 우선순위를 결정하는 기준

HRN 스케줄링

▪ HRN 스케줄링의 성능

도착 순서	도착 시간	작업 시간
P1	0	30
P2	3	18
P3	6	9

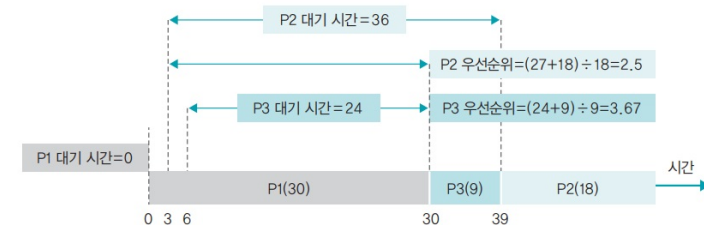


그림 4-19 HRN 스케줄링의 평균 대기 시간

© 2021 Computer System

© 2021 Computer System

HRN 스케줄링

HRN 스케줄링의 평가

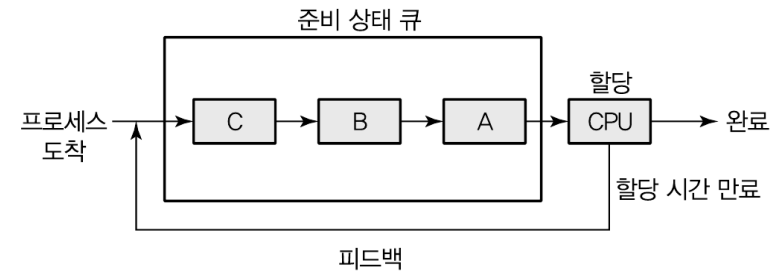
- 실행 시간이 짧은 프로세스의 우선순위를 높게 설정하면서도 대기 시간을 고려하여 야사 현상을 완화
- 대기 시간이 긴 프로세스의 우선순위를 높임으로써 CPU를 할당받을 확률을 높임
- 여전히 공평성이 위배되어 많이 사용되지 않음

© 2021 Computer System

라운드 로빈 스케줄링

라운드 로빈 스케줄링의 동작 방식

- 한 프로세스가 할당받은 시간(타임 슬라이스) 동안 작업을 하다가 작업을 완료하지 못하면 준비 큐의 맨 뒤로 가서 자기 차례를 기다리는 방식
- 선점형 알고리즘 중 가장 단순하고 대표적인 방식
- 프로세스들이 작업을 완료할 때까지 계속 순환하면서 실행

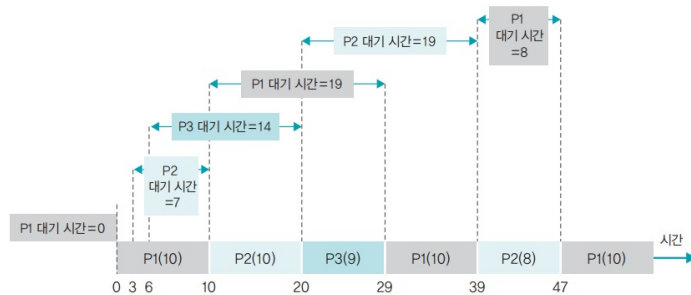


© 2021 Computer System

라운드 로빈 스케줄링

라운드 로빈 스케줄링의 성능

도착 순서	도착 시간	작업 시간
P1	0	30
P2	3	18
P3	6	9



© 2021 Computer System

라운드 로빈 스케줄링

타임 슬라이스의 크기와 문맥 교환

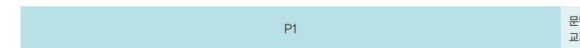
- 라운드 로빈 스케줄링이 효과적으로 작동하려면 문맥 교환에 따른 추가 시간을 고려하여 타임 슬라이스를 적절히 설정해야 함

타임 슬라이스가 큰 경우

- 하나의 작업이 끝난 뒤 다음 작업이 시작되는 것처럼 보여 FCFS 스케줄링과 비슷한 효과

타임 슬라이스가 작은 경우

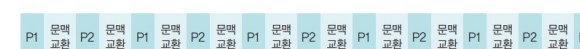
- 문맥 교환이 너무 자주 일어나 문맥 교환에 걸리는 시간이 실제 작업 시간보다 상대적으로 커지며, 문맥 교환에 많은 시간을 낭비하여 실제 작업을 못하는 문제가 발생



(a) 타임 슬라이스가 큰 경우



(b) 타임 슬라이스가 적당한 경우

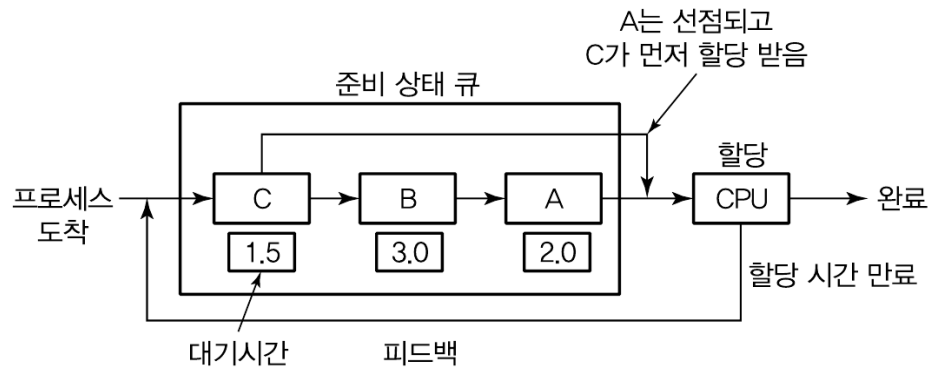


(c) 타임 슬라이스가 작은 경우

SRT(Short Remaining Time) 우선 스케줄링

■ SRT 스케줄링의 동작 방식

- 기본적으로 라운드 로빈 스케줄링을 사용하지만, CPU를 할당받을 프로세스를 선택할 때 남아 있는 작업 시간이 가장 적은 프로세스를 선택



© 2021 Computer System

SRT 우선 스케줄링

■ SRT 스케줄링의 성능

도착 순서	도착 시간	작업 시간
P1	0	30
P2	3	18
P3	6	9

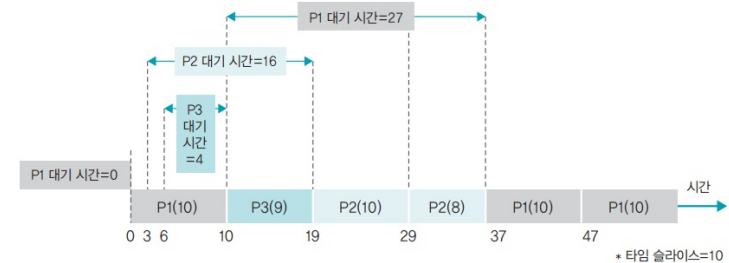


그림 4-24 SRT 스케줄링의 평균 대기 시간

© 2021 Computer System

SRT 우선 스케줄링

■ SRT 스케줄링의 평가

- 현재 실행 중인 프로세스와 큐에 있는 프로세스의 남은 시간을 주기적으로 계산하고, 남은 시간이 더 적은 프로세스와 문맥 교환을 해야 하므로 SJF 스케줄링에는 없는 작업이 추가됨
- 운영체제가 프로세스의 종료 시간을 예측하기 어렵고 아사 현상이 일어나기 때문에 잘 사용하지 않음

© 2021 Computer System

우선순위 스케줄링

■ 우선순위 스케줄링의 동작 방식

- 프로세스의 중요도에 따른 우선순위를 반영한 스케줄링 알고리즘

표 4-5 프로세스의 우선순위

도착 순서	도착 시간	작업 시간	우선순위
P1	0	30	3
P2	3	18	2
P3	6	9	1

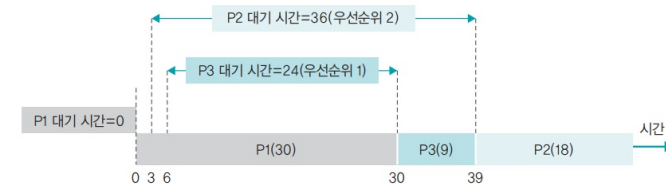


그림 4-25 FCFS 스케줄링에 우선순위를 적용한 결과

© 2021 Computer System

우선순위 스케줄링

- 고정 우선순위 알고리즘
 - 한 번 우선순위를 부여 받으면 종료될 때까지 우선순위가 고정
 - 단순하게 구현할 수 있지만 시시각각 변하는 시스템의 상황을 반영하지 못해 효율성이 떨어짐
- 변동 우선순위 알고리즘
 - 일정 시간마다 우선순위가 변하여 일정 시간마다 우선순위를 새로 계산하고 이를 반영
 - 복잡하지만 시스템의 상황을 반영하여 효율적인 운영 가능
- 우선순위 스케줄링의 평가
 - 준비 큐에 있는 프로세스의 순서를 무시하고 우선순위가 높은 프로세스에 먼저 CPU를 할당하므로 공정성을 위배하고 야사 현상을 일으킴
 - 준비 큐에 있는 프로세스의 순서를 무시하고 프로세스의 우선순위를 매번 바꿔야 하기 때문에 오버헤드가 발생하여 시스템의 효율성을 떨어뜨림

다단계 큐 스케줄링

- 다단계 큐 스케줄링의 동작 방식
 - 우선순위에 따라 준비 큐를 여러 개 사용하는 방식
 - 프로세스는 운영체제로부터 부여받은 우선순위에 따라 해당 우선순위의 큐에 삽입
 - 우선순위는 고정형 우선순위를 사용
 - 상단의 큐에 있는 모든 프로세스의 작업이 끝나야 다음 우선순위 큐의 작업이 시작됨

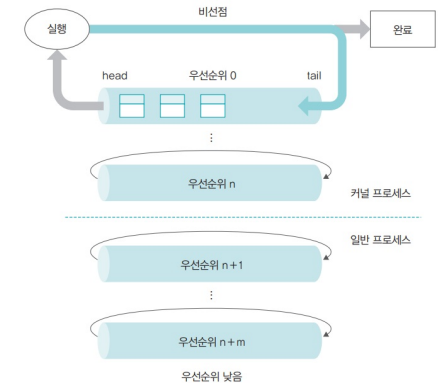
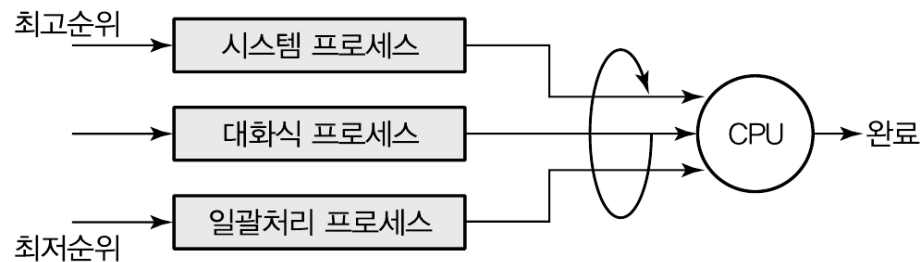


그림 4-26 다단계 큐 스케줄링의 동작

Multi-level Queue



다단계 피드백 큐 스케줄링

- 다단계 피드백 큐 스케줄링의 동작 방식
 - 프로세스가 CPU를 한 번씩 할당받아 실행될 때마다 프로세스의 우선순위를 낮춤으로써, 다단계 큐에서 우선순위가 낮은 프로세스의 실행이 연기되는 문제를 완화
 - 우선순위가 낮아진다고 할지라도 커널 프로세스가 일반 프로세스의 큐에 삽입되지는 않음
 - 우선순위에 따라 타임 슬라이스의 크기가 다름
 - 우선순위가 낮아질수록 CPU를 얻을 확률이 적어짐. 따라서 한번 CPU를 잡을 때 많이 작업하라고 낮은 우선순위의 타임 슬라이스를 크게 함.
 - 마지막 큐에 있는(우선순위가 가장 낮은) 프로세스는 무한대의 타임 슬라이스를 얻음
 - 마지막 큐는 들어온 순서대로 작업을 마치는 FCFS 스케줄링 방식으로 동작

다단계 피드백 큐 스케줄링

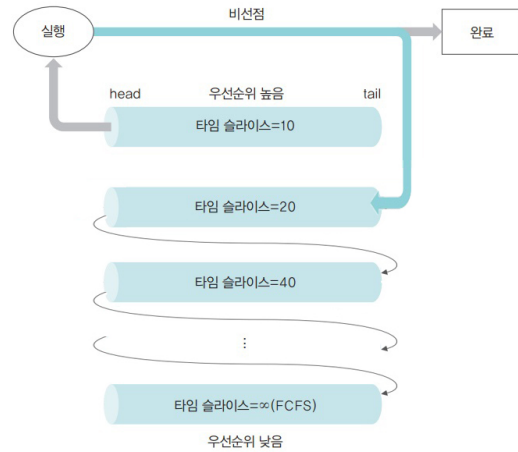


그림 4-27 다단계 피드백 큐 스케줄링의 동작

© 2021 Computer System

준비 상태의 다중 큐

준비 상태의 다중 큐

- 프로세스는 준비 상태에 들어올 때마다 자신의 우선순위에 해당하는 큐의 마지막에 삽입
- CPU 스케줄러는 우선순위가 가장 높은 큐(0번 큐)의 맨 앞에 있는 프로세스 6에 CPU 할당

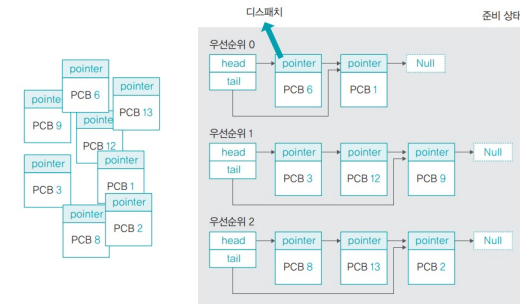


그림 4-9 준비 상태의 다중 큐

© 2021 Computer System

대기 상태의 다중 큐

다중 큐

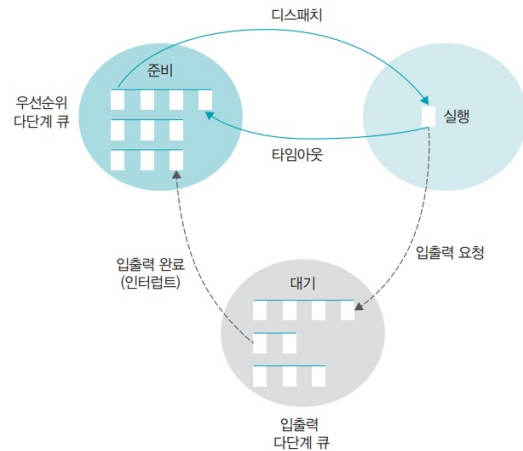


그림 4-11 프로세스 상태와 다중 큐

© 2021 Computer System

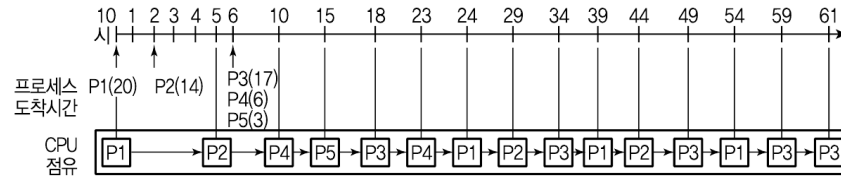
스케줄링기법의 계산 예제

프로세스	수행시간(초)	도착시간	우선순위
P1	20	10시	2
P2	14	10시 2초	3
P3	17	10시 6초	3
P4	6	10시 6초	1
P5	3	10시 6초	2

[조건] 문맥교환에 소요되는 시간은 0 으로 간주한다.

© 2021 Computer System

라운드 로빈

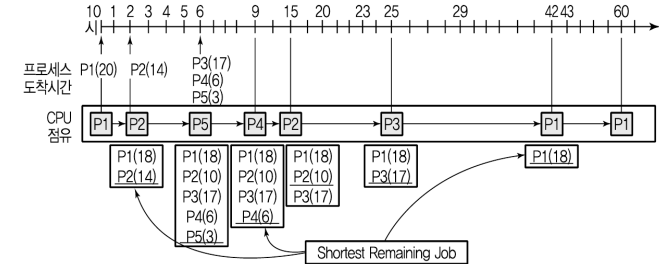


[Turnaround Time]

프로세스	Turnaround time
P1	10시 59초
P2	10시 49초
P3	10시 61초
P4	10시 24초
P5	10시 18초
평균 반환 시간	$(59+49+61+24+18)/5=10\text{시 } 42.2\text{초}$

© 2021 Computer System

SRT

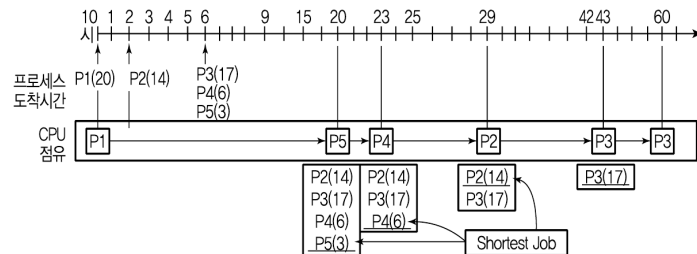


[Turnaround Time]

프로세스	Turnaround time
P1	10시 60초
P2	10시 25초
P3	10시 42초
P4	10시 15초
P5	10시 9초
평균 반환 시간	$(60+25+42+15+9)/5=10\text{시 } 30.2\text{초}$

© 2021 Computer System

SJF



[Turnaround Time]

프로세스	Turnaround time
P1	10시 20초
P2	10시 43초
P3	10시 60초
P4	10시 29초
P5	10시 23초
평균 반환 시간	$(20+43+60+29+23)/5=10\text{시 } 35\text{초}$

© 2021 Computer System

Context Switch 개념

가. Context Switching 정의

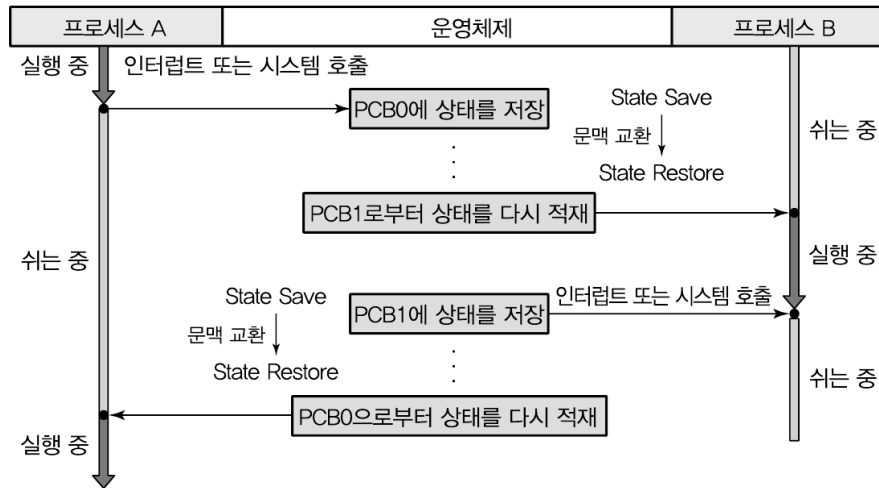
- 한 프로세스에서 다른 프로세스를 CPU가 새롭게 배당되는 교환과정을 문맥교환(Context switching)이라고 하며, 환경교환이라고도 함
- 프로세스들을 적절한 순서로 바꾸는 작업

나. Context Switching 발생 상황(프로세스 상태 천이 발생 시에 발생)

- 프로세스가 준비상태에서 실행상태로 변환될 때
- 프로세스가 실행상태에서 준비상태로 변환될 때 (Time slice Out)
- 프로세스가 실행상태에서 대기상태로 변환될 때 (I/O)
- 프로세스가 실행상태에서 종료상태로 변환될 때

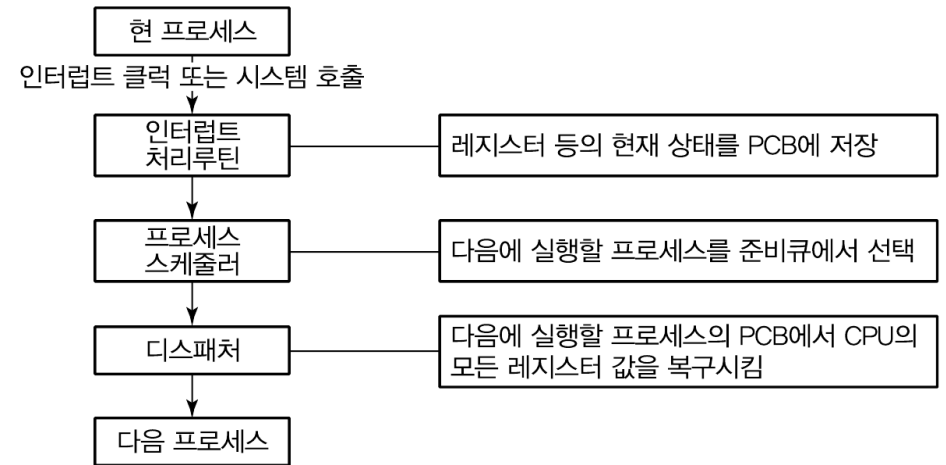
© 2021 Computer System

Context Switch 작동방식



© 2021 Computer System

Context Switching 문제점



© 2021 Computer System

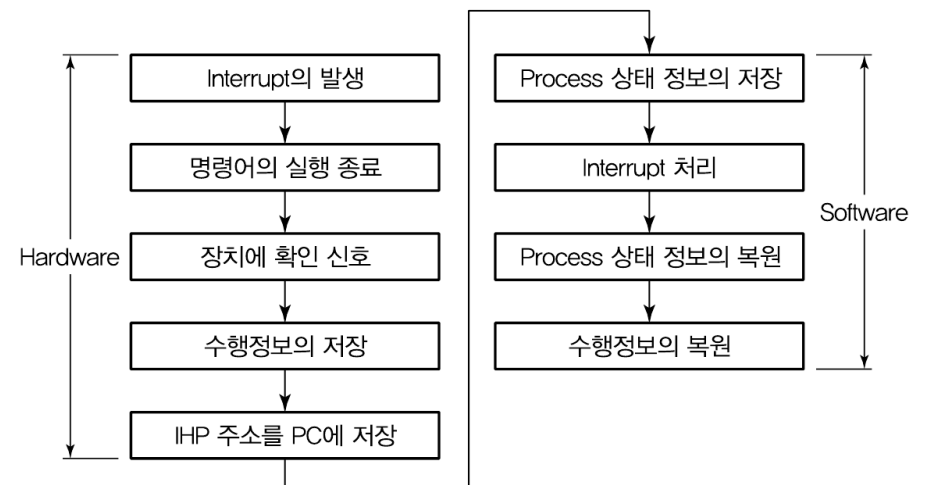
인터럽트의 개념

- 폴링
 - 입출력을 요청하면 운영체제가 주기적으로 입출력장치를 직접 확인해서 처리하는 방식
- 인터럽트
 - 이벤트 드리븐 방식과 마찬가지로 입출력을 요청하고 입출력이 완료되면 이벤트를 발생시켜 알림

구분	원인	설명
외부 인터럽트	컴퓨터 자체 내의 기계적인 문제	정전, 자료 전달과정의 오류, Timing Device에 의해 인 터럽트 유발
	컴퓨터 관리자의 의도적인 중단	Power Off, 작업 취소
	컴퓨터 주변 장치의 작동	속도가 빠른 Memory 상의 수행 도중에 속도가 낮은 Key Board, Mouse 등 주변장치의 입출력 요구
내부 인터럽트 (트랩이라고도 함)	프로그램상의 문제	보호된 Memory에 접근시도, 불법적인 Instruction 실행(Overflow, Stack Overflow, Divide By Zero 등)

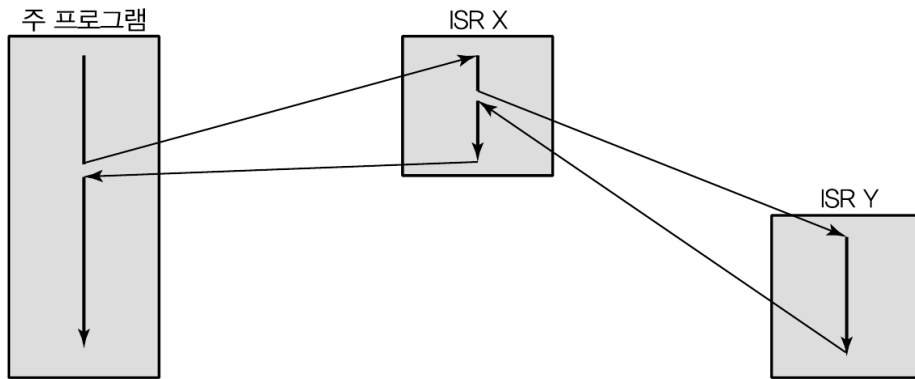
© 2021 Computer System

인터럽트 처리 방법



© 2021 Computer System

인터럽트 중첩



© 2021 Computer System

Summary



Image from <http://www.promotionalpro.com/blog/>

**Don't Guess....
Just ASK !**

© 2021 Computer System